

Label Noise in Just-In-Time Software Defect Prediction

Puneet Deshwani

2025MCS013

Under the supervision of Dr. Santosh Singh Rathore

Department of Computer Science and Engineering
Atal Bihari Vajpayee Indian Institute of Information Technology and Management, Gwalior

October 2026

Contents

1. Overview
2. Motivation
3. Related Works
4. Research Gaps
5. Objectives and Research Questions
6. Dataset
7. Proposed Methodology
8. References

Overview

- Just-in-time software defect prediction (JIT-SDP) predicts, at commit time, whether a code change is likely to introduce a defect, so that review and testing effort can be directed to risky changes [6].
- The models are trained on past commits. Each commit is described by change metrics and is labelled as clean or defect-introducing.
- These labels are not recorded in the repository. They are inferred by the SZZ algorithm, which traces the lines changed in a bug-fixing commit back to the commits that last modified them [11].
- Bug-fixing commits often contain unrelated changes (tangled commits), so SZZ produces wrong labels. Only 17% to 32% of the changed lines in bug-fixing commits contribute to the fix [5].

Overview

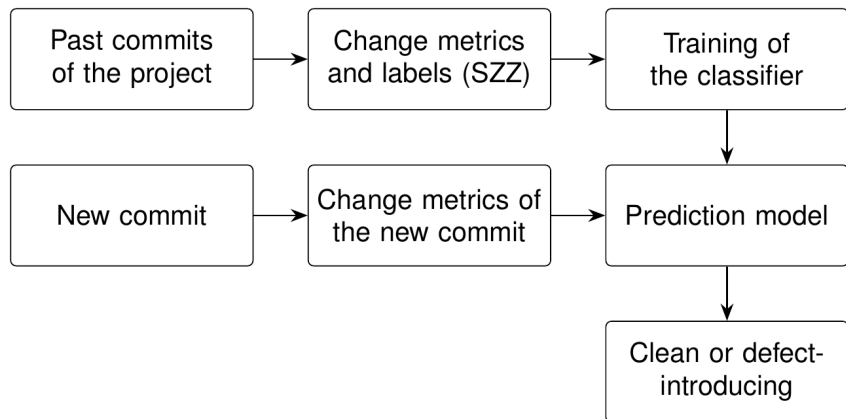


Figure 1: Workflow of JIT-SDP

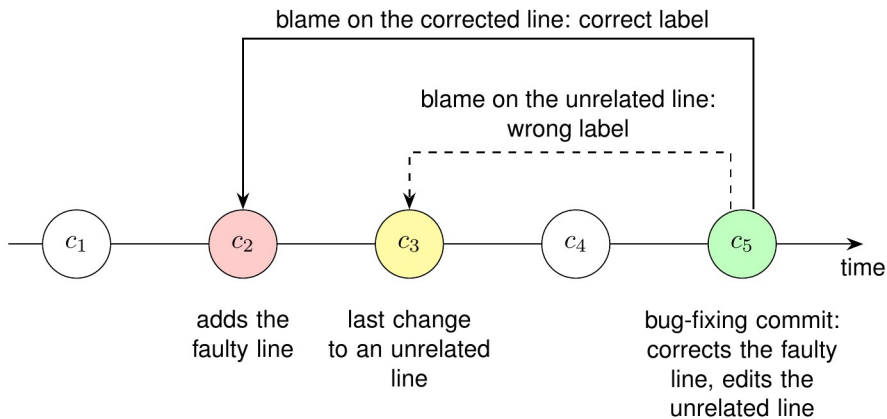


Figure 2: Labelling with SZZ and a wrong label caused by an unrelated change

Motivation

- JIT-SDP models are trained and tested on the same SZZ labels. A model is then rewarded for reproducing the errors of SZZ, and the effect of label noise on the reported performance is not visible.
- Random k-fold cross-validation is still widely used, although it trains the model on commits that are newer than the test commits [3].
- In practice the label of a commit arrives late (verification latency): a defect is recognised only when it is fixed, which can take months or years [1].
- It is therefore not known how much SZZ label noise costs a model that is evaluated in a realistic setting.

Related Works - 1

Title	Author	Dataset	Key Takeaways
When Do Changes Induce Fixes? (2005) [11]	Jacek Śliwerski	Mozilla and Eclipse	Proposes the SZZ algorithm: lines changed by a bug fix are traced back with annotate (blame) to find the bug-introducing changes
A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes (2017) [2]	Daniel A. da Costa	Ten open-source projects	Criteria for evaluating SZZ implementations; proposes MA-SZZ, which ignores meta-changes
Evaluating SZZ Implementations Through a Developer-Informed Oracle (2021) [10]	Giovanni Rosa	1,930 bug-fixing commits in which the developer names the introducing commit	Compares SZZ variants against the oracle; R-SZZ performs best (precision about 66%); releases PySZZ
A Fine-grained Data Set and Analysis of Tangling in Bug Fixing Commits (2022) [5]	Steffen Herbold	LLTC4J: manually labelled bug-fixing commits of Apache Java projects	Only 17% to 32% of the changed lines contribute to the bug fix; tangling adds noise to defect data
The Impact of Changes Mislabelled by SZZ on Just-in-Time Defect Prediction (2021) [4]	Yuanrui Fan	126,526 changes from ten Apache projects	Compares four SZZ variants with RA-SZZ as the reference; labels of AG-SZZ reduce performance significantly

Related Works - 2

Title	Author	Dataset	Key Takeaways
A Large-Scale Empirical Study of Just-in-Time Quality Assurance (2013) [6]	Yasutaka Kamei	Six open-source and five commercial projects	14 change metrics in five groups; 68% accuracy and 64% recall with logistic regression
Are Fix-Inducing Changes a Moving Target? (2018) [7]	Shane McIntosh	Qt and OpenStack (37,524 changes)	Properties of fix-inducing changes change over time; models lose predictive power as they age
Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction (2019) [1]	George G. Cabral	Ten GitHub projects	Online JIT-SDP with delayed labels (90-day waiting period); proposes ORB for evolving class imbalance
Deep Just-in-Time Defect Prediction: How Far Are We? (2021) [12]	Zhengran Zeng	Open-source projects with over 310,000 changes	Deep models do not consistently beat simple ones; LAPredict (logistic regression on added lines) is best in most cases
The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization (2022) [8]	Chao Ni	JIT-Defects4J: 21 Java projects, 27,319 commits	Dataset built from manually validated bug-fixing lines; proposes JIT-Fine

Research Gaps

- Noise of SZZ labels not measured over complete project histories
- Models tested against the same labels on which they are trained
- Effect of label noise not separated from the effect of the evaluation procedure
- Label noise not studied under verification latency
- Effect of SZZ noise on online learners not understood, and no remedy for this setting

Objectives

- **Objective 1:** To quantify the label noise introduced by SZZ and its variants, by comparing their labels with independent reference labels over complete project histories.
- **Objective 2:** To measure the effect of this noise on JIT-SDP models under a realistic evaluation that respects the order of the commits and the delay of the labels, and to separate it from the effect of the evaluation procedure.
- **Objective 3:** To identify how the noise affects an online learner, and to develop a noise-aware online learning approach that reduces its effect.

Research Questions

- **RQ1:** How much do the labels of SZZ variants disagree with one another and with independent reference labels, and what type of error does each variant make?
- **RQ2:** How does the choice of the label source affect the measured performance of JIT-SDP models under different evaluation settings?
- **RQ3:** Through which mechanism does label noise reduce the performance of an online learner, and which type of error is responsible?
- **RQ4:** Can an online learner be made less sensitive to SZZ label noise without access to clean labels?

Dataset

- JIT-Defects4J [8]: 27,319 commits from 21 Apache Java projects (2001 to 2019)
- 2,332 commits (8.54%) are defect-introducing;
5,453 commits are bug-fixing
- Reference labels: bug-fixing lines validated by at least three of four annotators, then traced back with git blame
- Features: the 14 change metrics of Kamei et al. [6]
- Selected because its labels do not come from the SZZ variants under study
- SZZ labels and label arrival times will be added in MTP II

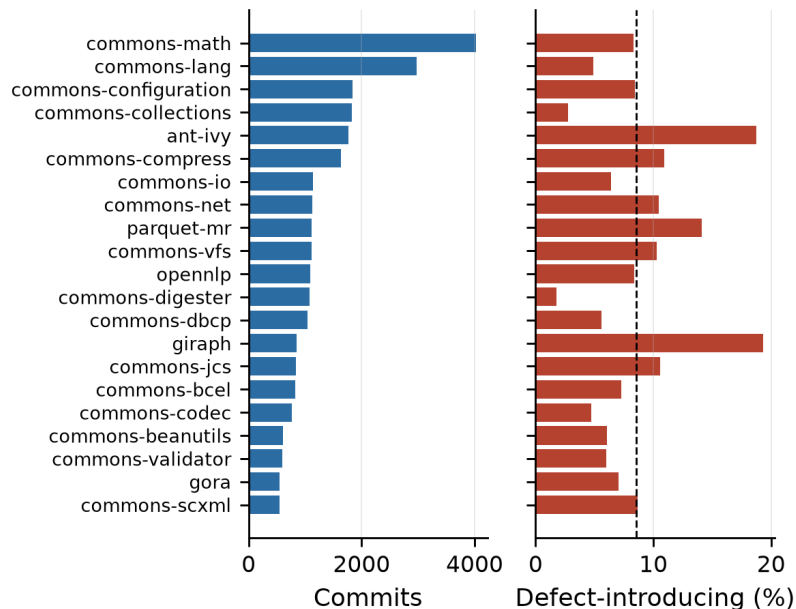


Figure 3: Commits and defect rate of each project

Proposed Methodology

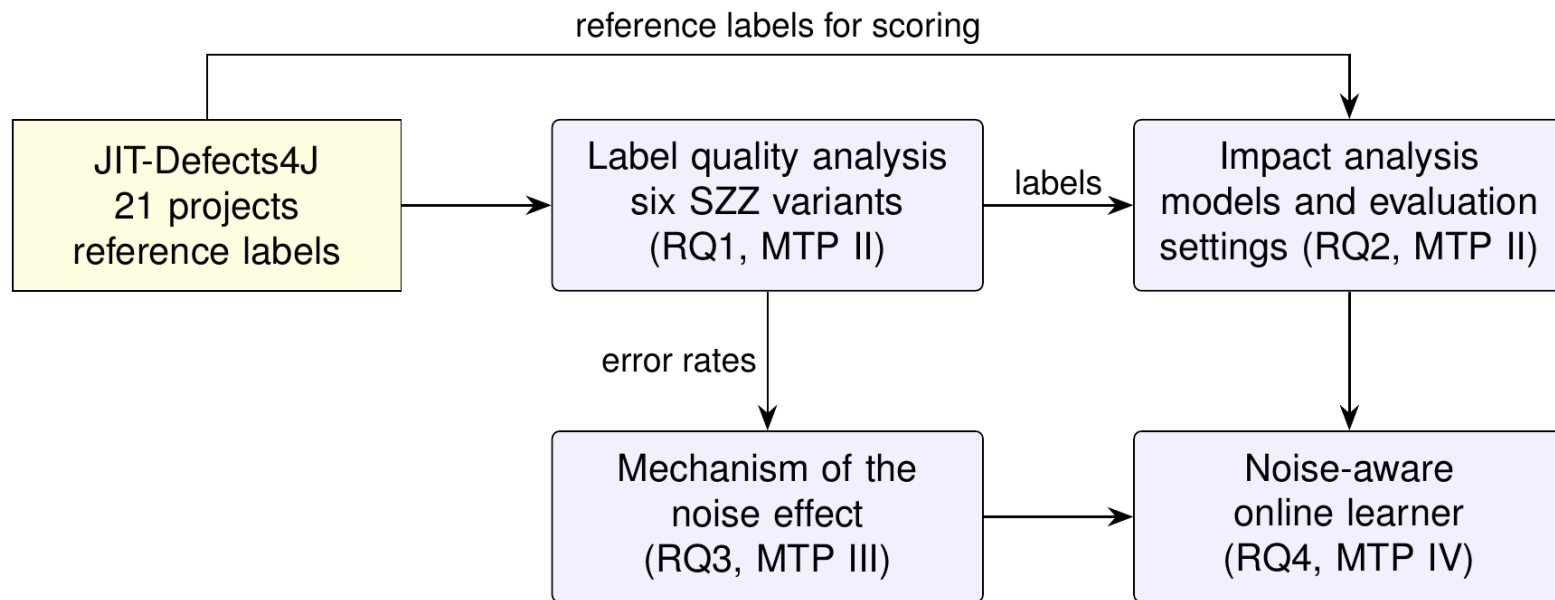


Figure 4: Planned studies and the research questions they address

Proposed Methodology

- **SZZ labels:** six variants (B-, AG-, MA-, L-, R- and RA-SZZ) will be run with PySZZ on the same 5,453 bug-fixing commits
- **Label quality:** precision, recall, false-alarm rate p_0 , miss rate p_1 and Cohen's κ against the reference labels
- **Models:** LAPredict (logistic regression on lines added) [12], a JITLine-based random forest with SMOTE [9], and ORB (online ensemble) [1]
- **Scoring:** against the reference labels, and against the training labels (self-scoring) for comparison
- **Statistics:** paired by project ($n = 21$), Wilcoxon signed-rank test, Hodges-Lehmann estimate with bootstrap confidence interval, Holm correction for multiple comparisons
- **Planned scale:** 3 models, 7 label sources, 4 evaluation settings, 10 seeds

Proposed Methodology: Evaluation Settings

(a) Random k -fold cross-validation (one fold shown)



(b) Chronological split and chronological-online



(c) Prequential evaluation with verification latency

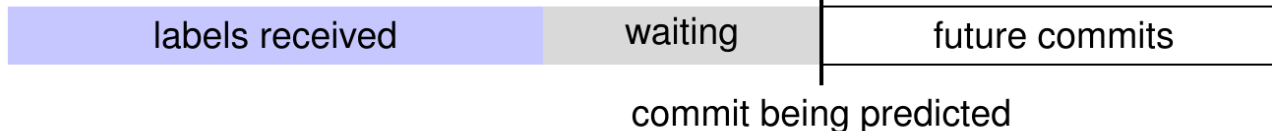


Figure 5: Evaluation settings. Commits of a project are ordered by time from left to right

References I

- [1] George G. Cabral et al. “Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction”. In: *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 666-676.
- [2] Daniel Alencar da Costa et al. “A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes”. In: *IEEE Transactions on Software Engineering* 43.7 (2017), pp. 641-657.
- [3] Davide Falessi et al. “On the Need of Preserving Order of Data When Validating Within-Project Defect Classifiers”. In: *Empirical Software Engineering* 25 (2020), pp. 4805-4830.
- [4] Yuanrui Fan et al. “The Impact of Changes Misabeled by SZZ on Just-in-Time Defect Prediction”. In: *IEEE Transactions on Software Engineering* 47.8 (2021), pp. 1559-1586.
- [5] Steffen Herbold et al. “A Fine-grained Data Set and Analysis of Tangling in Bug Fixing Commits”. In: *Empirical Software Engineering* 27.6 (2022), p. 125.
- [6] Yasutaka Kamei et al. “A Large-Scale Empirical Study of Just-in-Time Quality Assurance”. In: *IEEE Transactions on Software Engineering* 39.6 (2013), pp. 757-773.
- [7] Shane McIntosh and Yasutaka Kamei. “Are Fix-Inducing Changes a Moving Target? A Longitudinal Case Study of Just-in-Time Defect Prediction”. In: *IEEE Transactions on Software Engineering* 44.5 (2018), pp. 412-428.
- [8] Chao Ni et al. “The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2022, pp. 672-683.

References II

- [9] Chanathip Pornprasit and Chakkrit Tantithamthavorn. “JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction”. In: *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*. 2021, pp. 369-379.
- [10] Giovanni Rosa et al. “Evaluating SZZ Implementations Through a Developer-Informed Oracle”. In: *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. 2021.
- [11] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. “When Do Changes Induce Fixes?”. In: *Proceedings of the International Workshop on Mining Software Repositories (MSR)*. 2005, pp. 1-5.
- [12] Zhengran Zeng et al. “Deep Just-in-Time Defect Prediction: How Far Are We?”. In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2021.